



Security Audit Report

Tezoro

v1.0

May 15, 2026

Table of Contents

Table of Contents	2
License	4
Disclaimer	5
Introduction	6
Purpose of This Report	6
Codebase Submitted for the Audit	6
Methodology	7
Functionality Overview	7
How to Read This Report	8
Code Quality Criteria	9
Summary of Findings	10
Detailed Findings	12
1. Underdelivering strategy exits can block normal withdrawals	12
2. Withdrawals can erase stale balances and reopen cheap entry	12
3. Unswept base-asset rewards let new depositors capture prior yield	13
4. Stale trackedBalance misprices deposits and redemptions between reconcile calls	14
5. removeStrategy clears live strategy accounting and understates assets	15
6. Tiny interactions can raise highWaterMark without fees	16
7. Pause-insensitive liquidity estimates overstate ERC-4626 withdrawal limits	17
8. removeStrategy can orphan paused Aave positions and underprice shares	17
9. Early withdrawals can shift exit losses onto later users	18
10. Opaque claim and swap calldata can redirect rewards	19
11. Stale yield can let deposits bypass the cap	19
12. Unhealthy strategies can keep funds after de-risking	20
13. Fee hikes can overcharge old unreconciled yield	20
14. Reverting sub-vault views distort vault NAV and make healthy liquidity unreachable	21
15. Rewards module rotation can trap base asset	22
16. Missing baseToken validation strands CompoundV3Strategy funds in collateral mode	23
17. Paused strategies keep allowance and can still pull vault funds	23
18. Aave emergency withdrawal reverts and misclassifies recoverable funds as lost	24
19. Recalled strategies keep approval and may pull vault assets	25
20. Full-share FluidStrategy emergency redemption misclassifies recoverable funds as lost	25
21. Unrevoked sub-vault allowances allow frozen children to pull idle strategy assets	26
22. Redeem fallback underquotes shares and leaves sub-vault liquidity unreachable	27

23. Missing timelock expiry window enabling stale proposal execution	27
24. Missing aToken validation misreports balances and impairs recovery	28
25. Missing fToken asset validation defers deployment errors to runtime	29
26. Weak FluidStrategy health check misguides allocation and monitoring	29
27. Zero minAmountOut disables reward-swap slippage protection	30
28. Weak AaveV3Strategy health check misreports unsupplyable reserves as healthy	31
29. Weak MorphoBlueMultiStrategy health check misreports impaired markets	31
30. Improper health check logic for ERC4626MultiStrategy	32
31. CompoundV3Strategy health check misguides rebalancing into fully utilized Comet markets	32
32. Missing exit reconciliation can strand yield and underreport losses	33

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED “AS IS”, WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io/>
info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by the Arbitrum Foundation to perform a security audit of the Tezoro smart contracts.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	https://github.com/tezosproject/tezoro-autopilot
Commit	08b14b655d28af6f3822298104a07c1c40d86f23
Scope	The scope is restricted to v1.1 contracts. Already known issues described in the README are considered out of scope.
Fixes verified at commit	bc01a0c77f2b4015adbc6a4fda1ce45f6ce0e75b Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation

Functionality Overview

Tezoro is an ERC-4626 compliant vault that allocates user deposits across multiple DeFi lending protocols (Aave V3, Compound V3, Spark, Morpho Blue, Fluid, and any ERC-4626 vault).

A keeper rebalances allocations; a performance fee is charged on yield above a high water mark.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or denial of service.
Minor	A violation of common best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects, such as user experience or readability, but is not strictly necessary. This category may also include opinionated recommendations that the project team might not share.

The status of an issue can be one of the following: **Pending**, **Acknowledged**, **Partially Resolved**, or **Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium	The codebase integrates with multiple third-party contracts.
Code readability and clarity	Medium-High	-
Level of documentation	Medium-High	Codebase is well-documented and the architecture is clear.
Test coverage	Medium-High	Test coverage reported by <code>forge</code> coverage is 94.17%.

Summary of Findings

No	Description	Severity	Status
1	Underdelivering strategy exits can block normal withdrawals	Major	Resolved
2	Withdrawals can erase stale balances and reopen cheap entry	Major	Resolved
3	Unswept base-asset rewards let new depositors capture prior yield	Major	Resolved
4	Stale <code>trackedBalance</code> misprices deposits and redemptions between <code>reconcile</code> calls	Major	Resolved
5	<code>removeStrategy</code> clears live strategy accounting and understates assets	Major	Resolved
6	Tiny interactions can raise <code>highWaterMark</code> without fees	Minor	Resolved
7	Pause-insensitive liquidity estimates overstate ERC-4626 withdrawal limits	Minor	Resolved
8	<code>removeStrategy</code> can orphan paused Aave positions and underprice shares	Minor	Resolved
9	Early withdrawals can shift exit losses onto later users	Minor	Resolved
10	Opaque claim and swap calldata can redirect rewards	Minor	Resolved
11	Stale yield can let deposits bypass the cap	Minor	Resolved
12	Unhealthy strategies can keep funds after de-risking	Minor	Resolved
13	Fee hikes can overcharge old unreconciled yield	Minor	Resolved
14	Reverting sub-vault views distort vault NAV and make healthy liquidity unreachable	Minor	Resolved
15	Rewards module rotation can trap base asset	Minor	Resolved
16	Missing <code>baseToken</code> validation strands <code>CompoundV3Strategy</code> funds in collateral mode	Minor	Resolved
17	Paused strategies keep allowance and can still pull	Minor	Resolved

	vault funds		
18	Aave emergency withdrawal reverts and misclassifies recoverable funds as lost	Minor	Resolved
19	Recalled strategies keep approval and may pull vault assets	Minor	Resolved
20	Full-share FluidStrategy emergency redemption misclassifies recoverable funds as lost	Minor	Resolved
21	Unrevoked sub-vault allowances allow frozen children to pull idle strategy assets	Minor	Resolved
22	Redeem fallback underquotes shares and leaves sub-vault liquidity unreachable	Minor	Resolved
23	Missing timelock expiry window enabling stale proposal execution	Minor	Resolved
24	Missing aToken validation misreports balances and impairs recovery	Informational	Resolved
25	Missing fToken asset validation defers deployment errors to runtime	Informational	Resolved
26	Weak FluidStrategy health check misguides allocation and monitoring	Informational	Resolved
27	Zero minAmountOut disables reward-swap slippage protection	Informational	Resolved
28	Weak AaveV3Strategy health check misreports unsupplyable reserves as healthy	Informational	Resolved
29	Weak MorphoBlueMultiStrategy health check misreports impaired markets	Informational	Resolved
30	Improper health check logic for ERC4626MultiStrategy	Informational	Resolved
31	CompoundV3Strategy health check misguides rebalancing into fully utilized Comet markets	Informational	Resolved
32	Missing exit reconciliation can strand yield and underreport losses	Informational	Resolved

Detailed Findings

1. Underdelivering strategy exits can block normal withdrawals

Severity: Major

In `src/TezoroV1_1.sol:335-336`, the main vault reverts when a normal withdrawal recovers less than the exact requested amount.

In `src/TezoroV1_1.sol:949-961`, `_executeWithdrawal` tracks the actual amount returned by each strategy, so any short exit leaves `remaining` nonzero and causes that revert.

Some adapters also allow underdelivery by design, such as `src/strategies/FluidStrategy.sol:46-55`, which measures what the vault actually receives.

This means a normal user withdrawal can fail as soon as one integrated strategy returns slightly less than requested. That can happen with exit fees, rounding loss, delayed settlement, or liquidity limits.

Example: a user asks to withdraw 1,000 USDC. One strategy can only return 999.999 USDC because of an exit fee or rounding loss. The vault sees that the total recovered amount is still below the exact request, so it reverts the whole withdrawal with `WithdrawalFailed()`.

This is a liveness failure in the normal user exit path. Users may be unable to leave even though the vault still holds real value. This causes the intended functionality to be broken and users trying to withdraw their assets by burning fewer shares.

Recommendation

We recommend adding a bounded dust tolerance to normal user exits or blocking strategies that cannot return the exact requested amount in `V1_1`.

Status: Resolved

2. Withdrawals can erase stale balances and reopen cheap entry

Severity: Major

In `src/TezoroV1_1.sol:953-957`, the withdrawal path sets `trackedBalance[strategy]` to zero when the strategy returns more than the stale tracked amount.

This is unsafe when a strategy has unreconciled yield. A normal withdrawal can return more than the old tracked amount, but the strategy can still hold real assets after that.

The vault then pretends those remaining assets do not exist until the next `reconcile`.

Example: the vault thinks strategy A holds 100,000 USDC, but the real balance is 200,000 USDC because yield has not been reconciled yet. A user withdraws 150,000 USDC. The strategy returns 150,000 USDC, which is more than the stale tracked amount, so the vault sets `trackedBalance` to zero. In reality, strategy A still holds 50,000 USDC. A new depositor can then enter at a cheap price before the keeper calls `reconcile`.

Recommendation

We recommend refreshing the live strategy balance whenever `withdrawn > trackedBalance[strategy]`, instead of zeroing the tracked value and continuing.

Status: Resolved

3. Unswept base-asset rewards let new depositors capture prior yield

Severity: Major

In `src/RewardsModule.sol:176-184`, `sweepToVault` is the step that sends base-asset rewards from `RewardsModule` back into the vault. In `src/TezoroV1_1.sol:706-709`, `depositRewards` adds those rewards into vault accounting only when that sweep happens.

Because `totalAssets` in `src/TezoroV1_1.sol:193-200` does not include base-asset rewards that still sit in `RewardsModule`, deposits during that window are priced against stale assets.

This lets a depositor enter before the sweep and share rewards that were earned before the deposit.

For example, if Alice deposits 100,000 USDC, then 100,000 USDC of already-swapped rewards sit in `RewardsModule`, and Bob deposits 100,000 USDC before `sweepToVault`, the later sweep leaves both Alice and Bob worth about 150,000 USDC. Bob captures about 50,000 USDC of prior yield. This dilutes existing holders and breaks fair reward distribution.

Recommendation

We recommend minimizing or removing the post-swap, pre-sweep window. The same keeper workflow that creates base-asset proceeds should sweep them into the vault in the same transaction whenever possible.

If that is not possible, include pending base-asset rewards held in `RewardsModule` in deposit pricing so `totalAssets` reflects already-earned value before minting new shares.

Status: Resolved

4. Stale `trackedBalance` misprices deposits and redemptions between `reconcile` calls

Severity: Major

In the `TezoroV1_1` contract, `totalAssets` in `src/TezoroV1_1.sol:193-200` adds cached `trackedBalance` values instead of live strategy balances. `previewDeposit` in `src/TezoroV1_1.sol:257-260`, `maxWithdraw` and `maxRedeem` in `src/TezoroV1_1.sol:229-249`, and the deposit and redeem flows in `src/TezoroV1_1.sol:287-292` and `src/TezoroV1_1.sol:312-318` therefore, keep using stale NAV until `reconcile` refreshes `trackedBalance` in `src/TezoroV1_1.sol:640-649`.

If a strategy earns yield before `reconcile`, a new depositor can still mint shares at the old cheaper price and then share in that hidden gain after the next sync.

For example, if 1,000,000 USDC grows to 1,100,000 USDC off-vault and `reconcile` has not run yet, a later 1,000,000 USDC deposit is still priced against 1,000,000 USDC and captures part of the unreconciled 100,000 USDC.

If a strategy loses value before `reconcile`, an early redeemer can exit at an inflated price and leave more of the loss to remaining users. This creates a timing-dependent transfer of value between user cohorts and also makes performance fee timing and share previews incorrect between reconciliations. The transfer is bounded by the unreconciled balance delta and, for exits, by the liquidity the vault can still access.

Recommendation

We recommend refreshing healthy strategy balances before pricing deposits and withdrawals, or add a targeted pre-action reconciliation step for user entry and exit.

If live balance reads are too expensive or too brittle for every user action, operators should call `reconcile` before meaningful deposits or withdrawals after material strategy balance changes.

The `reconcile` comment should also not imply that calling `_accruePerformanceFee` closes this stale-NAV window.

Status: Resolved

5. `removeStrategy` clears live strategy accounting and understates assets

Severity: Major

In the `TezoroV1_1` contract, `removeStrategy` in `src/TezoroV1_1.sol:358-386` wraps `strategy.emergencyWithdraw` in a `try/catch`, computes recovered from the vault balance delta, and then unconditionally sets `trackedBalance[strategy] = 0`, removes the strategy from `strategies`, clears `isActiveStrategy[strategy]`, and revokes approval. This treats a partial or failed emergency exit as if the strategy is empty.

However, this is reachable in normal liquidity stress.

In the `CompoundV3Strategy` contract, `emergencyWithdraw` in `src/strategies/CompoundV3Strategy.sol:66-82` returns 0 when pool liquidity is exhausted.

In the `FluidStrategy` contract, `emergencyWithdraw` in `src/strategies/FluidStrategy.sol:57-63` can revert when redeem cannot be serviced.

In both cases, the external position can still hold funds, but `totalAssets` in `src/TezoroV1_1.sol:193-200` no longer counts them because the strategy is removed and its tracked balance is zero.

This puts the vault in an incorrect state, can understate share price and underpay users who exit before the admin manually restores the strategy and calls `reconcile`, and relies on `StrategyRemovalFundsLost` being noticed off-chain.

Recommendation

We recommend clearing the strategy state only after `emergencyWithdraw` proves that the position is empty. If the call returns less than the tracked amount or reverts, keep the strategy active in a frozen or distressed state, preserve its accounting, and let the admin retry reconcile, `recallToIdle`, or final removal after liquidity returns.

Status: Resolved

6. Tiny interactions can raise `highWaterMark` without fees

Severity: Minor

In `src/TezoroV1_1.sol:1017-1043`, the `_accruePerformanceFee` function checks whether the share price is above `highWaterMark`, calculates the performance fee, mints fee shares only if the rounded fee is non-zero, and then still updates `highWaterMark` to the new share price.

This lets the benchmark move up even when the vault collects no fee. A user can trigger this through `deposit`, `mint`, `withdraw`, or `redeem`. Here, external gain means any increase in `totalAssets` that does not mint shares. This can happen through a direct token donation to the vault, rewards added through `depositRewards`, or strategy yield recognized by `reconcile`.

For example, an adversary can donate 10 USDC directly to the vault and then call `deposit(1)`. The 10 USDC donation increases `totalAssets`, but it does not mint shares.

When `deposit(1)` runs, `_accruePerformanceFee` sees a much higher share price. Since the fee rounds down to zero in this path, the vault collects no fee, but `highWaterMark` still moves up. Later profits are then compared against this inflated benchmark.

This reduces protocol fee income. Small gains can be added into the benchmark without paying fees first. Later profits below the inflated `highWaterMark` pay no performance fee until the share price moves above it again.

Recommendation

We recommend updating `highWaterMark` only when the vault actually charges a non-zero performance fee, or keeping small unpaid gains in separate accounting until they are large enough to mint fee shares. We recommend rejecting deposits that mint zero shares as well, because that path makes the problem worse.

Status: Resolved

7. Pause-insensitive liquidity estimates overstate ERC-4626 withdrawal limits

Severity: Minor

In `src/TezoroV1_1.sol:980-990`, `_availableLiquidity` aggregates the idle asset balance held by the vault with each strategy-reported `availableLiquidity` value and uses that total to bound `maxWithdraw` and `maxRedeem`. However, this logic trusts direct adapter liquidity estimates that only compare position size with pool cash. In `AaveV3Strategy`, `availableLiquidity` derives the result from the `aToken` balance and reserve cash, and in `CompoundV3Strategy`, `availableLiquidity` derives the result from the `comet` balance and base-asset cash without incorporating the upstream pause conditions that determine whether `withdraw` can actually succeed.

Consequently, `maxWithdraw` and `maxRedeem` can return a positive amount during protocol pause or emergency states even though the corresponding withdrawal call reverts at execution time. This weakens the ERC-4626 withdrawal-limit semantics of the vault and can mislead users, frontends, and integrators about presently executable liquidity precisely when conservative reporting matters most.

Recommendation

We recommend making `availableLiquidity` in `AaveV3Strategy` and `CompoundV3Strategy`, which return 0 whenever the underlying protocol disables withdrawals and otherwise deriving the value from the same conditions that govern a successful `withdraw`. If a uniform per-adapter pause check is not available, the vault should treat affected strategies conservatively when computing `maxWithdraw` and `maxRedeem` so those functions only expose amounts that are presently executable.

Status: Resolved

8. `removeStrategy` can orphan paused Aave positions and underprice shares

Severity: Minor

The `emergencyWithdraw` function in `src/strategies/AaveV3Strategy.sol:63-68` calls `pool.withdraw` without checking the Aave reserve configuration returned by `getReserveData`.

The same blind spot exists in `availableLiquidity` in `src/strategies/AaveV3Strategy.sol:75-79` and `isHealthy` in `src/strategies/AaveV3Strategy.sol:81-84`, which only read token balances. As a result, the strategy cannot distinguish normal liquidity from a paused reserve. When Aave pauses a reserve, the underlying tokens still sit at the `aToken` address, so the strategy still looks liquid and healthy, but `pool.withdraw` reverts.

However, `removeStrategy` in `src/TezoroV1_1.sol:358-381` catches that revert, then still sets `trackedBalance[strategy] = 0` and `isActiveStrategy[strategy] = false`.

Since `totalAssets` in `src/TezoroV1_1.sol:193-200` only sums idle assets and `trackedBalance`, the live Aave position drops out of accounting while it remains inside the strategy.

This can orphan funds and immediately reduce the share price for remaining depositors.

Recommendation

We recommend reading the reserve configuration before reporting health, liquidity, or calling `pool.withdraw`.

If a reserve is paused, `removeStrategy` should either revert or keep the strategy active and its `trackedBalance` unchanged until withdrawal succeeds. The strategy should also treat frozen reserves as unhealthy or deposit-frozen so the vault does not assume new Aave deposits are available.

Status: Resolved

9. Early withdrawals can shift exit losses onto later users

Severity: Minor

In `src/strategies/ERC4626MultiStrategy.sol:117-166`, `ERC4626MultiStrategy` accepts the actual amount received from child vault exits, even when that amount is lower than requested. In `src/TezoroV1_1.sol:943-972`, the main vault keeps pulling from later strategies until the current user is paid.

This means one user can exit first and get paid from healthy strategies even if one child vault exits short. The shortfall is not charged to the first user. It is pushed onto the users who stay in the vault.

Example: Alice and Bob each deposit 1,000,000 USDC. One child vault later exits with a small haircut. Alice withdraws first. The vault fills Alice's gap by taking more from a healthy strategy. Bob stays in the vault and is left with less backing than before, even though the loss was realized during Alice's exit.

Recommendation

We recommend changing exit accounting so a child-vault shortfall is handled directly instead of being silently filled with value from unrelated healthy strategies.

Status: Resolved

10. Opaque claim and swap calldata can redirect rewards

Severity: Minor

`executeClaim` in `src/RewardsModule.sol:105-114` only whitelists `target` and `selector`, then forwards raw `calldata` to the claim target. `swap` in `src/RewardsModule.sol:141-161` only whitelists the router and checks that `tokenOut` equals `baseAsset`, then forwards raw `routerData` and relies on the module's before-and-after base-asset balance. Neither path validates recipient-style fields in the forwarded `calldata` or enforces that proceeds return to the `RewardsModule`.

This is only safe when the downstream integration is proof-bound and the verified claim or swap semantics already fix the payout destination. For the currently intended URD or Merkl-style claim flows, this may not be a concrete bug because the recipient appears to be proof-bound.

However, the contract does not enforce that invariant. If a current or future whitelisted target or router exposes a free recipient or to parameter, an `admin` or `keeper` can still call an approved selector and redirect claimed rewards or `swap` proceeds away from the module. We classify this issue as minor since it depends on a trusted role or a later unsafe whitelist addition, but it can still divert reward tokens from the intended compounding flow.

Recommendation

We recommend only whitelisting claim `targets` and `routers` whose payout destination is fixed to the rewards module or is bound inside verified proof data. If broader integrations are needed, add target-specific wrappers or decode and validate recipient fields in `data` and `routerData` before making the external call. This trust boundary should also be documented explicitly.

Status: Resolved

11. Stale yield can let deposits bypass the cap

Severity: Minor

In `src/TezoroV1_1.sol:193-200`, `totalAssets` uses stale `trackedBalance`. In `src/TezoroV1_1.sol:209-221`, `maxDeposit` and `maxMint` enforce `depositCap` from that same stale value. In `src/TezoroV1_1.sol:640-649`, strategy gains are only recognized when `reconcile` runs.

This means the vault can show free cap room even when strategy yield has already pushed the real asset value above the intended limit. A user can deposit before the keeper reconciles, and the vault ends up above the configured cap later.

Example: the vault cap is 150,000 USDC. Alice deposits 100,000 USDC. The strategy then earns 80,000 USDC, but the keeper has not reconciled yet. Bob still sees 50,000 USDC of

free cap room and deposits it. After `reconcile`, the vault holds 230,000 USDC, even though the cap was 150,000 USDC.

Recommendation

We recommend treating `depositCap` as soft until `reconcile` runs, or using live strategy value for cap checks if the cap must be strict.

Status: Resolved

12. Unhealthy strategies can keep funds after de-risking

Severity: Minor

In `src/TezoroV1_1.sol:576-579`, `_rebalance` skips a strategy completely when `isHealthy` returns false. The same loop later contains both the `deposit` branch and the `withdrawal` branch in `src/TezoroV1_1.sol:595-629`.

This means the health check blocks not only new deposits, but also withdrawals used for de-risking. Once a strategy becomes unhealthy, the keeper cannot use `rebalance` to move funds away from it even if that strategy is still withdrawable.

Example: strategy A becomes unhealthy, but it still has 50,000 USDC of available liquidity. The keeper sets its target allocation to 0 and wants to move funds to strategy B.

`rebalance` skips strategy A entirely because `isHealthy` is false, so the 50,000 USDC stays in the risky strategy until `recallToIdle` is called.

Recommendation

We recommend using the health check only for new deposits. Withdrawals that reduce risk should still be allowed when liquidity exists.

Status: Resolved

13. Fee hikes can overcharge old unreconciled yield

Severity: Minor

In `src/TezoroV1_1.sol:755-764`, `setPerformanceFee` calls `_accruePerformanceFee` before changing the fee rate. In `src/TezoroV1_1.sol:640-649`, `reconcile` is the step that makes the stale strategy yield visible to the vault.

This means old yield can stay hidden at the old fee rate, then become visible only after the fee rate is increased. When the keeper later reconciles and collects fees, that old yield can be charged at the new, higher rate.

Example: the strategy already earned 20,000 USDC, but the keeper has not called reconcile yet. The admin raises the fee from 15% to 30%. After reconcile, the old 20,000 USDC is finally visible and gets charged at the new 30% rate, even though it was earned before the fee hike.

Recommendation

We recommend refreshing strategy balances first, then accruing fees at the old rate, and only after that changing the fee rate.

Status: Resolved

14. Reverting sub-vault views distort vault NAV and make healthy liquidity unreachable

Severity: Minor

In `src/strategies/ERC4626MultiStrategy.sol:196-215`, the `balanceOf` and `availableLiquidity` views aggregate the wrapper idle balance with every approved sub-vault by directly calling each child `balanceOf`, `convertToAssets`, and `maxWithdraw`. In lines 202-205 and 213-214, these external reads execute without per-sub-vault fault isolation, so one reverting third-party sub-vault causes the entire strategy-level view to revert even when the wrapper still holds idle assets or other healthy sub-vault positions.

However, this failure propagates differently through the vault and creates two downstream problems. When `balanceOf` reverts, `reconcile` skips the strategy and leaves `trackedBalance` unchanged, which preserves stale accounting. When `availableLiquidity` reverts, both `_availableLiquidity` and `_executeWithdrawal` skip the entire parent strategy, so the vault never reaches the wrapper's own fault-tolerant `withdraw` path that already attempts to continue past broken sub-vaults.

Consequently, healthy liquidity inside the wrapper can become unavailable to vault-mediated withdrawals, and `maxWithdraw` and `maxRedeem` can under-report assets that the wrapper can still retrieve from idle funds or unaffected sub-vaults. At the same time, if the unreadable sub-vault is actually impaired and enough idle or other strategy liquidity remains available, stale `trackedBalance` can keep `totalAssets` overstated, allowing early redeemers to exit against inflated NAV while new depositors receive too few shares until operators pause or remove the strategy.

Recommendation

We recommend isolating each sub-vault read inside `balanceOf` and `availableLiquidity` with per-sub-vault `try/catch` and exposing an explicit incomplete-read signal to the vault whenever any child read fails. Make `availableLiquidity` best-effort so one reverting child cannot hide healthy wrapper liquidity from withdrawal and preview paths, and ensure `reconcile` does not silently preserve or update `trackedBalance` as authoritative when the strategy balance is incomplete or unreadable.

Status: Resolved

15. Rewards module rotation can trap base asset

Severity: Minor

In `src/TezoroV1_1.sol:749-752`, `setRewardsModule` switches the trusted rewards module address immediately.

In `src/TezoroV1_1.sol:706-709`, `depositRewards` only accepts calls from the current module.

In `src/RewardsModule.sol:176-181`, the old module can only return base asset through `sweepToVault`.

In `src/RewardsModule.sol:220-223`, `rescueToken` blocks rescuing the base asset.

This means the base asset can get stuck in the old rewards module if the admin rotates to a new module before the old one is empty. After the rotation, the old module can no longer sweep the base asset into the vault, and it still cannot rescue the base asset.

Example: the old rewards module holds 50,000 USDC. Admin points the vault to a new rewards module. The old module now fails in `sweepToVault` because it is no longer the trusted module, and it also fails in `rescueToken` because the token is the base asset. The 50,000 USDC stays stuck in the old module.

Recommendation

We recommend requiring the old rewards module to sweep before rotation, or adding an explicit migration path that can move the base asset from the old module into the vault or the new module.

Status: Resolved

16. Missing baseToken validation strands CompoundV3Strategy funds in collateral mode

Severity: Minor

In `src/strategies/CompoundV3Strategy.sol:34-46`, the `CompoundV3Strategy` constructor stores arbitrary `asset_` and `comet_` inputs and approves the selected `Comet` to pull the selected asset, but it never verifies that `ICompoundV3Comet(comet_).baseToken() equals asset_`.

However, in lines 49-52, `deposit` always calls `comet.supply(asset, amount)`, and in lines 54-92, `withdraw`, `emergencyWithdraw`, `balanceOf`, and `availableLiquidity` all assume the position is base supply by relying on `comet.balanceOf` and base-token liquidity. If the configured `Comet` accepts `asset_` as collateral instead of as its base token, deposits succeed in collateral mode while the adapter accounting and exit logic no longer match the actual position.

Consequently, the strategy can accept vault funds and hold them inside `Comet` while reporting 0 through `balanceOf` and `availableLiquidity`, which causes `withdraw` and `emergencyWithdraw` to short-circuit without recovering the deposited asset. This issue primarily stems from deployment misconfiguration rather than an unprivileged runtime exploit, but the failure mode remains severe because the vault lacks a normal adapter-mediated recovery path after funds enter collateral mode. If `asset_` is unsupported by the selected `Comet`, the first deposit reverts, so the dangerous case is a mismatched `Comet` that still accepts the asset as collateral.

Recommendation

We recommend enforcing the asset-market pairing in the `CompoundV3Strategy` constructor by checking that `ICompoundV3Comet(comet_).baseToken() equals asset_` and reverting otherwise. Keep any deployment-time validation in tooling, but duplicate it on-chain so a mismatched asset and `Comet` configuration fails fast instead of silently creating an unusable collateral-only position.

Status: Resolved

17. Paused strategies keep allowance and can still pull vault funds

Severity: Minor

In `pauseStrategy` in `src/TezoroV1_1.sol:391-395`, the vault only sets `pausedStrategies[strategy] = true`. The unlimited allowance that is granted to each strategy in `addStrategy` in `src/TezoroV1_1.sol:344-353` stays live until `removeStrategy` in `src/TezoroV1_1.sol:381-386`.

The strategy pattern uses that allowance to pull assets from the vault with `safeTransferFrom`, for example, in `src/strategies/AaveV3Strategy.sol:48-52`.

As a result, `pauseStrategy` stops the normal deposit flow, but it does not remove the paused strategy's spend authority. If a strategy is malicious or becomes compromised, it can still pull idle vault funds, and the guardian pause does not fully contain the incident.

Recommendation

We recommend revoking the strategy allowance inside `pauseStrategy`, and restoring it only when the strategy is intentionally unpaused or otherwise allowed to receive deposits again. This makes a pause, removing the strategy's ability to pull assets from the vault.

Status: Resolved

18. Aave emergency withdrawal reverts and misclassifies recoverable funds as lost

Severity: Minor

In `src/strategies/AaveV3Strategy.sol:63-68`, the `emergencyWithdraw` function reads the strategy `aToken` balance and, when that balance is nonzero, calls `pool.withdraw(asset, type(uint256).max, vault)` to redeem the full position directly to the vault.

However, unlike the normal `withdraw` path, this logic does not cap the request to `availableLiquidity`, so it ignores the reserve's current liquid underlying and assumes the full position is immediately withdrawable even during high utilization.

Consequently, `emergencyWithdraw` can revert even when the strategy still holds a valid Aave position and a partial withdrawal succeeds. This primarily affects admin recovery rather than routine user exits, but it becomes operationally significant because `removeStrategy` invokes `strategy.emergencyWithdraw`, measures the recovered amount, emits `StrategyRemovalFundsLost` when recovery is short, and then zeros `trackedBalance`. A temporarily illiquid position therefore appears lost in vault accounting and strategy removal flows even though the assets remain deposited and recoverable when liquidity returns.

Recommendation

We recommend capping `emergencyWithdraw` to current liquid reserves before calling `pool.withdraw`, mirroring the conservative behavior already used by the normal `withdraw` path and by `CompoundV3Strategy`.

Compute the withdrawable amount through `availableLiquidity`, withdraw only the lesser of the strategy `aToken` balance and that amount, and treat any remaining

utilization-constrained balance as temporarily illiquid rather than as an unrecoverable loss during strategy removal or other admin recovery flows.

Status: Resolved

19. Recalled strategies keep approval and may pull vault assets

Severity: Minor

In the `TezoroV1_1` contract, `recallToIdle` in `src/TezoroV1_1.sol:423-457` sets the target allocation to 0 and freezes deposits, but it does not clear the strategy allowance. The vault grants `type(uint256).max` approval in `addStrategy` in `src/TezoroV1_1.sol:344-355`, and only clears it in `removeStrategy` in `src/TezoroV1_1.sol:358-388`.

This leaves the external `strategy` contract inside the vault's token trust boundary even after it is recalled. If the strategy implementation is buggy, malicious, upgradeable, or otherwise compromised, it may still pull idle assets and later deposits from the vault until it is removed. This weakens the emergency isolation path and can lead to loss of user funds. We classify this issue as minor since it depends on a faulty approved strategy, but the vault should fully revoke trust during recall.

Recommendation

We recommend clearing the asset allowance in `recallToIdle` with `IERC20(asset()).forceApprove(address(strategy), 0)` after the withdrawal attempts finish. Restore approval only when the strategy is explicitly allowed to receive deposits again. This makes the recall path actually isolate the strategy.

Status: Resolved

20. Full-share `FluidStrategy` emergency redemption misclassifies recoverable funds as lost

Severity: Minor

In `src/strategies/FluidStrategy.sol:46-62`, the `withdraw` path caps exits to `availableLiquidity` through `fToken.maxWithdraw`, while `emergencyWithdraw` redeems the strategy's entire `fToken` share balance directly to the vault. However, this emergency path bypasses the current `Fluid` withdrawal cap, so `fToken.redeem` reverts whenever the full share position exceeds the amount presently withdrawable from the Liquidity layer even though a partial exit remains available.

Consequently, routine user withdrawals remain mostly unaffected because the normal `withdraw` path stays conservative, but admin recovery flows become unnecessarily brittle. `removeStrategy` invokes `emergencyWithdraw` directly, so a temporarily liquidity-limited position can trigger `StrategyRemovalFundsLost` and be treated as fully unrecoverable

even when part of the position is still withdrawable, and `recallToIdle` retains the same fragility in its emergency fallback.

Recommendation

We recommend making `emergencyWithdraw` best-effort by capping the exit to the current Fluid limit before redeeming. Use `fToken.maxRedeem` and redeem only that share amount, or mirror the normal path by reading `fToken.maxWithdraw` and calling `fToken.withdraw` for the capped asset amount, so temporary Liquidity-layer limits do not cause a full emergency-withdraw failure or a false loss classification.

Status: Resolved

21. Unrevoked sub-vault allowances allow frozen children to pull idle strategy assets

Severity: Minor

In `src/strategies/ERC4626MultiStrategy.sol:266-336`, `addSubVault` grants each approved child `type(uint256).max` allowance over the strategy asset, `freezeSubVaultDeposits` and `recallSubVault` only set `depositFrozenSubVaults`, and only `removeSubVault` clears the approval with `forceApprove(subVault, 0)`.

However, freezing or recalling a sub-vault does not revoke its existing token approval, so these containment paths only block the strategy's own `allocate` flow and do not limit what the approved child can still do through the already granted ERC-20 allowance. As a result, a frozen or recalled child is not fully quarantined at the approval boundary, even though the operator workflow suggests that it is isolated.

Consequently, if a still-approved sub-vault becomes malicious or compromised after operators freeze or recall it, that child can call `transferFrom` and pull any idle asset balance that later accumulates in `ERC4626MultiStrategy` from fresh vault deposits or from redemptions out of other sub-vaults. This exposure persists until `removeSubVault` executes and zeroes the allowance, so an intermediate quarantine step can leave recoverable idle funds exposed to a child that operators already regard as unsafe.

Recommendation

We recommend revoking the asset allowance whenever deposits to a sub-vault are frozen, at minimum in `recallSubVault` and ideally also in `freezeSubVaultDeposits`, and restoring approval only when operators explicitly unfreeze the child or immediately before the next `allocate`. This keeps the operational meaning of a frozen or recalled sub-vault aligned with the actual token-approval boundary and prevents a quarantined child from pulling later idle balances.

Status: Resolved

22. Redeem fallback underquotes shares and leaves sub-vault liquidity unreachable

Severity: Minor

In `src/strategies/ERC4626MultiStrategy.sol:151-158`, the `withdraw` function of `ERC4626MultiStrategy` falls back from `withdraw` to `redeem` for sub-vaults that do not support `withdraw` by converting the requested asset amount with `convertToShares` and redeeming that share amount. However, `convertToShares` is a deposit-side quote that rounds down the shares corresponding to an asset amount, while a redeem fallback that aims to satisfy a withdrawal target needs a withdraw-side quote such as `previewWithdraw`, which rounds up the shares required to obtain the requested assets.

Consequently, the fallback can redeem too few shares and deliver less than the intended amount, or compute zero shares for a nonzero withdrawal target and skip a sub-vault that still has redeemable liquidity. This primarily affects compatibility and liveness rather than direct asset loss, but it can leave healthy redeem-only sub-vault liquidity unreachable through ordinary vault-mediated withdrawals until an administrator uses a broader recovery path.

Recommendation

We recommend replacing the fallback share quote with a withdraw-side calculation, such as `previewWithdraw` before calling `redeem`, and capping that share amount against `maxRedeem` if additional defensive compatibility is needed. This keeps the fallback aligned with ERC-4626 rounding semantics and prevents redeem-only liquidity from being skipped or under-withdrawn because of round-down quoting.

Status: Resolved

23. Missing timelock expiry window enabling stale proposal execution

Severity: Minor

In `src/TezoroV1_1.sol:812-824`, the timelock mechanism implemented via `proposeTimelock` and `_consumeTimelockIfActive` enforces only a lower-bound delay (`readyTimestamp`) before execution, but does not define an upper-bound expiry. Once a proposal becomes executable (i.e., `block.timestamp >= readyTimestamp`) and remains unexecuted, it can be executed indefinitely.

This design deviates from standard timelock patterns, such as those used in OpenZeppelin's `TimelockController`, which enforce a bounded execution window via a grace period. The absence of such a constraint introduces a stale-context execution vulnerability, where operations approved under previous assumptions may be executed long after the system's state or threat model has changed.

A malicious actor or inattentive administrator could exploit this by executing outdated proposals that are no longer valid or safe.

Recommendation

We recommend introducing a bounded execution window by implementing a `GRACE_PERIOD` following the `readyTimestamp`.

Status: Resolved

24. Missing `aToken` validation misreports balances and impairs recovery

Severity: Informational

In `src/strategies/AaveV3Strategy.sol:33-45`, the `AaveV3Strategy` constructor accepts `asset_`, `pool_`, and `aToken_` as independent inputs, stores them, and approves the selected `pool`, but it never verifies that `aToken_` matches the reserve token returned by `pool.getReserveData(asset_)`.

However, `deposit` supplies funds through the configured `pool` and `asset`, while `balanceOf`, `availableLiquidity`, `isHealthy`, `harvest`, and the zero-balance gate in `emergencyWithdraw` all rely on the injected `aToken`, so the adapter assumes these inputs describe the same reserve without enforcing that invariant on-chain.

Consequently, a deployment with the correct `asset` and `pool` but the wrong `aToken` can still create a real Aave position while the adapter reads accounting and liquidity from an unrelated token. In that state, `balanceOf` and `availableLiquidity` can report 0, `harvest` can claim rewards against the wrong reserve token, and `emergencyWithdraw` can short-circuit even though the actual position remains in Aave.

This issue primarily stems from deployment misconfiguration rather than an unprivileged runtime exploit, but the failure mode is silent and can leave funds behind broken reporting and recovery paths until operators intervene.

Recommendation

We recommend enforcing the reserve-token pairing in the `AaveV3Strategy` constructor by querying `pool.getReserveData(asset_)` and reverting unless the returned `aTokenAddress` equals `aToken_`. If available in deployment tooling, retain the off-chain validation as well, but keep the on-chain check so an inconsistent `asset`, `pool`, and `aToken` configuration fails immediately instead of deploying a strategy whose accounting and recovery logic target the wrong reserve.

Status: Resolved

25. Missing `fToken` asset validation defers deployment errors to runtime

Severity: Informational

In `src/strategies/FluidStrategy.sol:28-38`, the `FluidStrategy` constructor accepts `asset_`, `fToken_`, and `vault_`, checks only for zero addresses, stores the configured asset and `fToken`, and grants the selected `fToken` unlimited approval over `asset_`.

However, the constructor never verifies that the underlying asset reported by `fToken` matches `asset_`, even though `deposit` in lines 41-43 later pulls `asset` from the vault and forwards that amount into `fToken.deposit` under the assumption that both assets are identical.

Consequently, a mismatched `fToken` does not fail fast at deployment and instead remains deployable until the first live allocation reaches `deposit`. In the normal case, the call reverts atomically because the strategy holds and approves `asset_` while the configured `fToken` attempts to pull a different underlying asset, so funds do not become silently misaccounted. Even so, the adapter enters production in a broken configuration, shifts a simple wiring error into runtime operations, and delays detection until capital allocation begins.

Recommendation

We recommend enforcing the asset pairing in the constructor by querying the configured `fToken` underlying and reverting unless it matches `asset_`. Add a dedicated mismatch error if desired, and allow the approval and deposit flow to proceed only after this invariant holds, so an incorrect `fToken` selection fails immediately during deployment instead of during the first live `deposit`.

Status: Resolved

26. Weak `FluidStrategy` health check misguides allocation and monitoring

Severity: Informational

In `src/strategies/FluidStrategy.sol:77-80`, `FluidStrategy.isHealthy` returns `true` whenever `fToken.totalAssets` is greater than zero, so it treats global market non-emptiness as the strategy health signal.

However, this predicate does not indicate whether this strategy accepts new deposits, whether its own position is withdrawable, or whether the market is otherwise a sensible allocation target. Because the vault uses `isHealthy` as the rebalance deposit gate and also surfaces the same flag through `getAllocationStatus`, the method reports unrelated market activity as strategy health.

Consequently, a Fluid market with third-party deposits but poor current exit liquidity or other usability constraints still appears healthy, so operators and automation continue treating it as a valid destination for fresh capital. Conversely, a fresh but otherwise valid empty market appears unhealthy and does not receive its initial allocation. This issue primarily distorts allocation and monitoring rather than directly causing asset loss, but it makes the vault health signal unreliable for the decisions it gates.

Recommendation

We recommend replacing the `fToken.totalAssets` check with a health predicate that matches the deposit decision that `isHealthy` gates, such as a positive `maxDeposit` capability or a protocol-specific pause or liveness check. If operators also need current exit visibility, expose that through `availableLiquidity` or a separate status signal instead of overloading `isHealthy` with a proxy that does not reflect strategy usability.

Status: Resolved

27. Zero `minAmountOut` disables reward-swap slippage protection

Severity: Informational

In `src/RewardsModule.sol:134-156`, the `swap` function approves a whitelisted router, executes opaque `routerData`, measures `amountOut` from the change in the `baseAsset` balance, and enforces slippage only by reverting in line 156 when `amountOut` is less than `minAmountOut`.

However, the function never rejects `minAmountOut` equal to 0, so the caller can disable the only explicit on-chain output floor and still satisfy the check with any nonreverting execution, including one that returns only dust.

Consequently, a benign but buggy keeper integration can silently remove slippage protection by submitting a default zero value, allowing a poor route, thin liquidity, or adverse MEV execution to consume most or all pending reward value before `sweepToVault`. This issue remains inside the `RewardsModule` trust boundary because user principal does not leave the vault through this path, but it still weakens a meaningful safety control and turns a simple operator error into avoidable reward loss.

Recommendation

We recommend rejecting zero `minAmountOut` values in `swap` and reverting before approving or calling the router when the supplied slippage floor is zero. If stronger protection is desired, add an admin-configurable minimum output bound per reward token or route so a keeper-supplied value cannot reduce slippage protection to a meaningless nonzero amount.

Status: Resolved

28. Weak AaveV3Strategy health check misreports unsupplyable reserves as healthy

Severity: Informational

In `src/strategies/AaveV3Strategy.sol:81-84`, `isHealthy` in `AaveV3Strategy` returns `true` whenever the configured `aToken` holds some underlying asset liquidity.

However, this check only measures current reserve cash and does not inspect the Aave reserve state that determines whether new supply is allowed, such as inactive, frozen, or paused conditions available through `pool.getReserveData`. As a result, the strategy can report itself as healthy even when the reserve no longer accepts fresh deposits.

Consequently, the vault can continue to treat the strategy as a valid allocation target during rebalance and report it as healthy through `getAllocationStatus` even after Aave rejects supply into that reserve. Deposit attempts then revert inside `strategy.deposit`, the vault silently skips them through `try/catch`, target allocations remain unmet, gas is wasted, and operators receive an overly optimistic status signal for a reserve whose withdrawal side may still function while its deposit side is disabled.

Recommendation

We recommend making `isHealthy` reflect actual Aave depositability instead of only reserve cash by reading `pool.getReserveData`, decoding the reserve status flags, and returning `false` whenever the reserve is inactive, frozen, or paused. If a liquidity check remains desirable, combine it with that configuration test so the strategy reports healthy only when the reserve both accepts new deposits and has positive available liquidity.

Status: Resolved

29. Weak MorphoBlueMultiStrategy health check misreports impaired markets

Severity: Informational

In `MorphoBlueMultiStrategy.isHealthy` in `src/strategies/MorphoBlueMultiStrategy.sol:222-229`, the strategy returns `true` when any approved Morpho market has non-zero `totalSupplyShares`.

This is only a global market liveness check. It does not check whether this strategy's own position is still solvent or whether funds are currently withdrawable. The vault uses this value as the only automated deposit gate in `TezoroV1_1.rebalance` in `src/TezoroV1_1.sol:574-580`.

Because of this, the vault can keep adding funds to an impaired market. If a market has bad debt, `totalSupplyShares` can stay non-zero while suppliers are diluted. If a market is near

full utilization, `totalSupplyShares` also stays non-zero while usable liquidity is effectively zero.

In both cases, the keeper continues depositing until an admin manually calls `freezeMarketDeposits`, which can increase losses or trap more assets in an illiquid market

Recommendation

We recommend making `isHealthy` check strategy-level safety instead of only `totalSupplyShares`. It should return false when a configured market has bad debt, when the strategy has no meaningful withdrawable liquidity, or when deposits are frozen.

Status: Resolved

30. Improper health check logic for `ERC4626MultiStrategy`

Severity: Informational

In `src/strategies/ERC4626MultiStrategy.sol`, the `ERC4626MultiStrategy.isHealthy` function determines strategy health by checking whether any sub-vault reports `totalAssets() > 0`. This logic is flawed because `IERC4626.totalAssets` reflects the aggregate assets held by the vault across all depositors, rather than the assets attributable to the strategy itself or the vault's available liquidity.

As a result, for any active ERC-4626-compliant vault, `totalAssets` will almost always return a non-zero value as long as at least one depositor exists. This causes `isHealthy` to consistently return true under normal operating conditions, regardless of the actual health, solvency, or liquidity state of the strategy.

Since this function acts as the sole on-chain safeguard gating `_rebalance` operations, a malicious actor or degraded external protocol could exploit this design flaw.

Recommendation

We recommend redesigning the health check mechanism to rely on strategy-specific and security-relevant indicators rather than global vault metrics.

Status: Resolved

31. `CompoundV3Strategy` health check misguides rebalancing into fully utilized Comet markets

Severity: Informational

In `src/strategies/CompoundV3Strategy.sol:88-97`, `availableLiquidity` in lines 88-92 caps withdrawable assets to the lesser of the strategy `comet.balanceOf` position and the current asset balance held by Comet, while `isHealthy` in lines 95-97 returns `true` whenever `comet.isSupplyPaused` and `comet.isWithdrawPaused` are both `false`.

However, `isHealthy` does not consider the current base-token liquidity that the adapter already uses to model immediate exit capacity. A fully utilized Comet market therefore, remains healthy for allocation purposes even when `availableLiquidity` is zero or materially reduced.

Consequently, the vault can continue depositing idle funds into a market whose immediate base liquidity is exhausted because `_rebalance` uses `isHealthy` as its deposit gate. This does not overstate `maxWithdraw` or `maxRedeem`, since the withdrawal-preview path remains conservative through `availableLiquidity`, but it does make the health signal too permissive for allocation and monitoring and can increase exposure to temporary withdrawal bottlenecks while `getAllocationStatus` still reports the strategy as healthy.

Recommendation

We recommend either incorporating current base liquidity into `isHealthy` or adding a separate rebalance-side guard that skips new deposits when `availableLiquidity` is zero or falls below a defined threshold. Keep the withdrawal-preview path unchanged, but make the signal that governs fresh allocations reflect both protocol liveness and present exit capacity so a fully utilized Comet market is not treated as a suitable destination for additional capital.

Status: Resolved

32. Missing exit reconciliation can strand yield and underreport losses

Severity: Informational

In `removeStrategy` in `src/TezoroV1_1.sol:358-370`, the vault reads `trackedBalance[strategy]` and calls `strategy.emergencyWithdraw` without first refreshing the strategy balance. In `recallToIdle` in `src/TezoroV1_1.sol:423-458`, the vault again relies on the stale tracked value and calls `strategy.withdraw(tracked)` before any balance sync. The contract already contains reconciliation logic in `reconcile` in `src/TezoroV1_1.sol:640-655`, but these exit paths do not use it.

If a strategy accrues yield after the last accounting update, `recallToIdle` may withdraw only the old tracked amount and leave extra assets behind. `removeStrategy` may also remove the strategy and zero its accounting without recognizing unrecovered accrued yield. This can underreport losses and reduce the assets recovered for users during admin emergency actions.

Recommendation

We recommend refactoring the balance sync logic into an internal helper and run it before `removeStrategy` and `recallToIdle`. The sync should best-effort read `strategy.balanceOf` and update `trackedBalance[strategy]`, but it should ignore failures so a broken strategy still cannot block removal or recall. The refreshed value should then be used for withdrawal sizing and loss accounting.

Status: Resolved